ASSIGNMENT: 12.1

NAME: Sri Harsha

HALLTICKET NUMBER: 2303A52159

BATCH: 36

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.

o Ask AI to include time complexity and space complexity in the function docstring.

o Verify the generated code with test cases.

• Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.

```python
def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorithm.

    Time Complexity:
    - Best Case: O(n log n)
    - Average Case: O(n log n)
    - Worst Case: O(n log n)

    Space Complexity:
    - O(n) auxiliary space for temporary arrays used during merging.
    - Recursive stack space: O(log n)

    Parameters:
    arr (list): List of elements to be sorted.

    Returns:
    list: New sorted list in ascending order.
    """

    if len(arr) <= 1:
        return arr

    # Find middle index
    mid = len(arr) // 2

    # Divide the array into two halves
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    # Merge the sorted halves
    return merge(left, right)


def merge(left, right):
    result = []
    i = j = 0

    # Compare elements from left and right arrays
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])

    return result
```

```
# -------- Test Cases --------

def test_merge_sort():
    print("Running Test Cases...\n")

    test_cases = [
        [38, 27, 43, 3, 9, 82, 10],
        [5, 2, 9, 1, 5, 6],
        [],
        [1],
        [10, 9, 8, 7, 6],
        [3, 3, 3, 3],
        [-5, -10, 0, 5, 2]
    ]

    for i, case in enumerate(test_cases, 1):
        sorted_arr = merge_sort(case)
        print(f"Test Case {i}:")
        print("Original:", case)
        print("Sorted:  ", sorted_arr)
        print("-" * 30)


# Run tests
test_merge_sort()
```

```
Running Test Cases...

Test Case 1:
Original: [38, 27, 43, 3, 9, 82, 10]
Sorted:   [3, 9, 10, 27, 38, 43, 82]
------------------------------
Test Case 2:
Original: [5, 2, 9, 1, 5, 6]
Sorted:   [1, 2, 5, 5, 6, 9]
------------------------------
Test Case 3:
Original: []
Sorted:   []
------------------------------
Test Case 4:
Original: [1]
Sorted:   [1]
------------------------------
Test Case 5:
Original: [10, 9, 8, 7, 6]
Sorted:   [6, 7, 8, 9, 10]
------------------------------
Test Case 6:
Original: [3, 3, 3, 3]
Sorted:   [3, 3, 3, 3]
------------------------------
Test Case 7:
Original: [-5, -10, 0, 5, 2]
Sorted:   [-10, -5, 0, 2, 5]
------------------------------
```

Explaination : The merge_sort() function divides the list into smaller halves recursively until each part has one element, then the merge() function combines these parts by comparing elements to produce a sorted list in ascending order.

Complexity: Time Complexity = O(n log n), Space Complexity = O(n).

Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

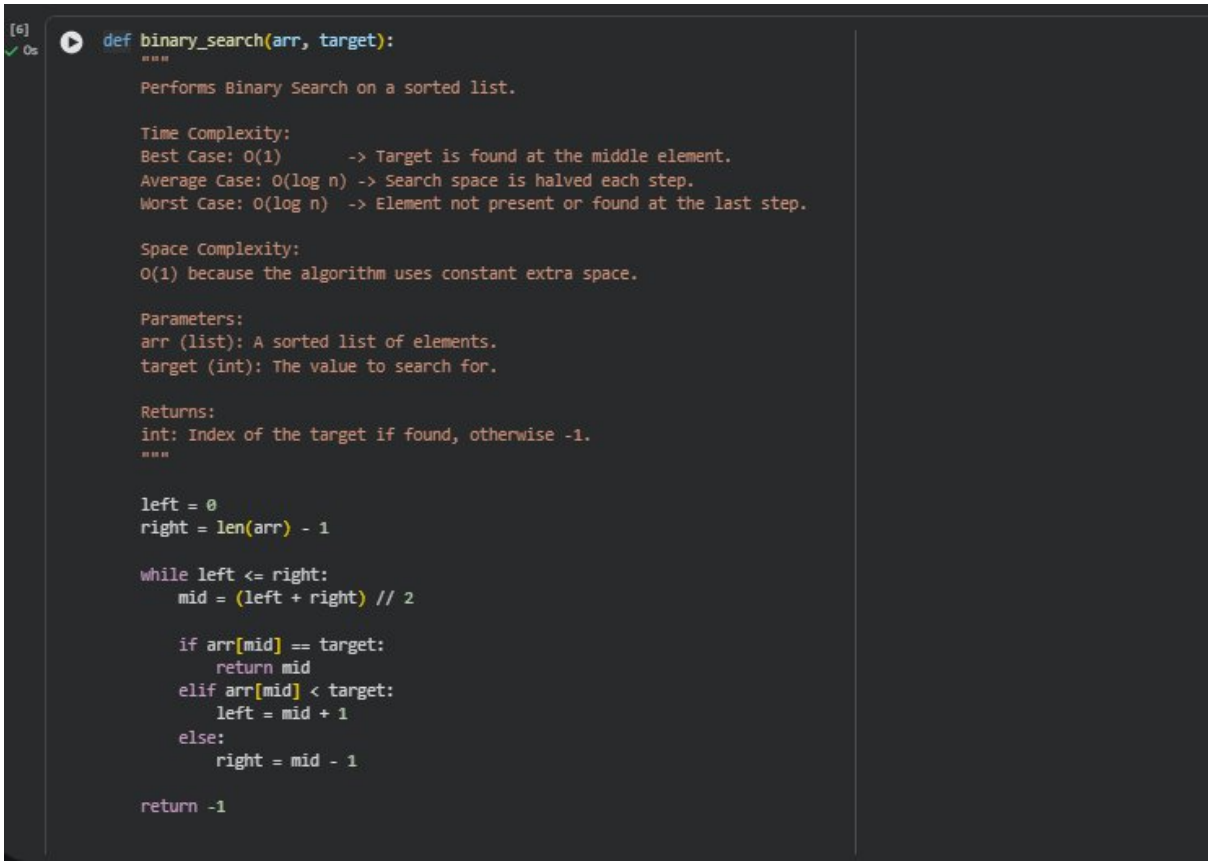
o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

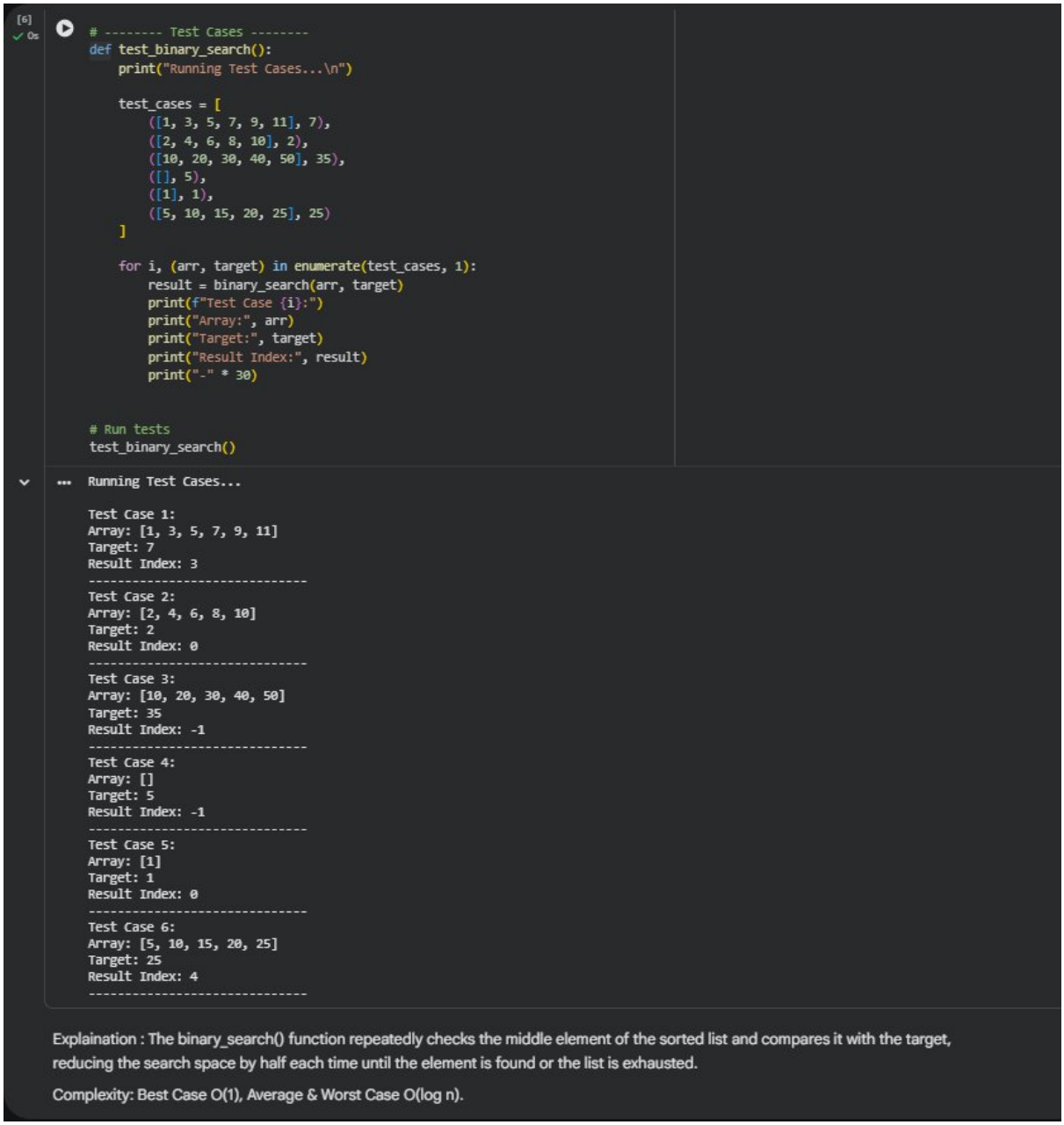
o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.



```
[6]
✓ 0s
def binary_search(arr, target):
    """
    Performs Binary Search on a sorted list.

    Time Complexity:
    Best Case: O(1)       -> Target is found at the middle element.
    Average Case: O(log n) -> Search space is halved each step.
    Worst Case: O(log n)  -> Element not present or found at the last step.

    Space Complexity:
    O(1) because the algorithm uses constant extra space.

    Parameters:
    arr (list): A sorted list of elements.
    target (int): The value to search for.

    Returns:
    int: Index of the target if found, otherwise -1.
    """

    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```



```
[6]
✓ 0s
# -------- Test Cases --------
def test_binary_search():
    print("Running Test Cases...\n")

    test_cases = [
        ([1, 3, 5, 7, 9, 11], 7),
        ([2, 4, 6, 8, 10], 2),
        ([10, 20, 30, 40, 50], 35),
        ([], 5),
        ([1], 1),
        ([5, 10, 15, 20, 25], 25)
    ]

    for i, (arr, target) in enumerate(test_cases, 1):
        result = binary_search(arr, target)
        print(f"Test Case {i}:")
        print("Array:", arr)
        print("Target:", target)
        print("Result Index:", result)
        print("-" * 30)


# Run tests
test_binary_search()
```

```
Running Test Cases...

Test Case 1:
Array: [1, 3, 5, 7, 9, 11]
Target: 7
Result Index: 3
------------------------------
Test Case 2:
Array: [2, 4, 6, 8, 10]
Target: 2
Result Index: 0
------------------------------
Test Case 3:
Array: [10, 20, 30, 40, 50]
Target: 35
Result Index: -1
------------------------------
Test Case 4:
Array: []
Target: 5
Result Index: -1
------------------------------
Test Case 5:
Array: [1]
Target: 1
Result Index: 0
------------------------------
Test Case 6:
Array: [5, 10, 15, 20, 25]
Target: 25
Result Index: 4
------------------------------
```

Explaination : The binary_search() function repeatedly checks the middle element of the sorted list and compares it with the target, reducing the search space by half each time until the element is found or the list is exhausted.

Complexity: Best Case O(1), Average & Worst Case O(log n).

Task Description #3 (Real-Time Application – Inventory Management System)

• Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

• Task:

o Use AI to suggest the most efficient search and sort algorithms for this use case.

o Implement the recommended algorithms in Python.

o Justify the choice based on dataset size, update frequency, and performance requirements.

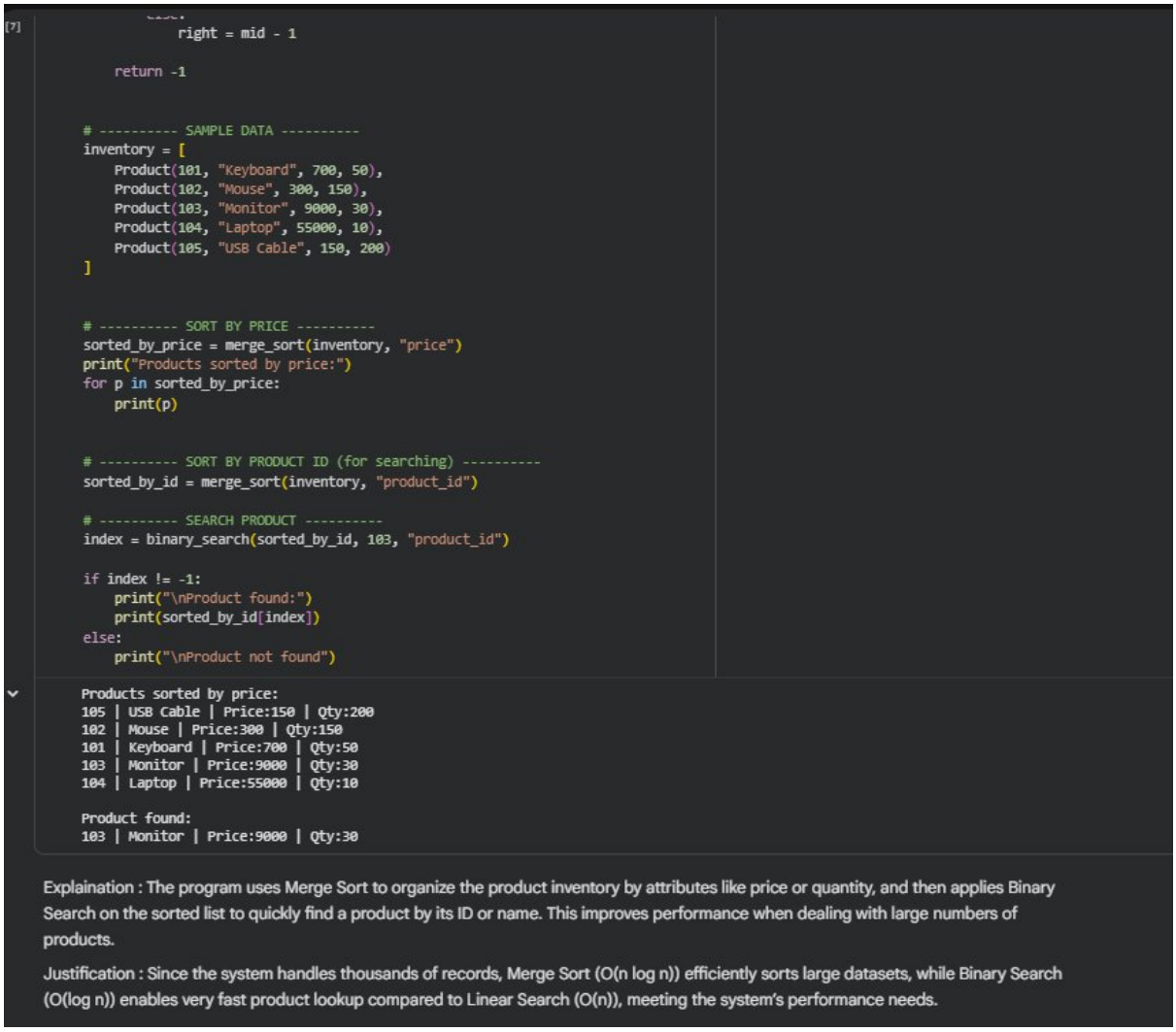
• Expected Output:

o A table mapping operation → recommended algorithm → justification.

o Working Python functions for searching and sorting the inventory.

[7]

```python
# Recommended Algorithms
# Searching Algorithm: Binary Search : Time Complexity : Best Case: O(1), Average Case: O(log n), Worst Case: O(log n)
#Sorting Algorithm: Merge Sort : Time Complexity : Best / Average / Worst: O(n log n)

# Product class
class Product:
    def __init__(self, product_id, name, price, quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.quantity = quantity

    def __repr__(self):
        return f"{self.product_id} | {self.name} | Price:{self.price} | Qty:{self.quantity}"


# ---------- MERGE SORT ----------
def merge_sort(products, key):
    if len(products) <= 1:
        return products

    mid = len(products) // 2
    left = merge_sort(products[:mid], key)
    right = merge_sort(products[mid:], key)

    return merge(left, right, key)


def merge(left, right, key):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if getattr(left[i], key) <= getattr(right[j], key):
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


# ---------- BINARY SEARCH ----------
def binary_search(products, target, key):
    left = 0
    right = len(products) - 1

    while left <= right:
        mid = (left + right) // 2
        value = getattr(products[mid], key)

        if value == target:
            return mid
        elif value < target:
            left = mid + 1
        else:
            right = mid - 1
```



```
            right = mid - 1

    return -1


# ---------- SAMPLE DATA ----------
inventory = [
    Product(101, "Keyboard", 700, 50),
    Product(102, "Mouse", 300, 150),
    Product(103, "Monitor", 9000, 30),
    Product(104, "Laptop", 55000, 10),
    Product(105, "USB Cable", 150, 200)
]


# ---------- SORT BY PRICE ----------
sorted_by_price = merge_sort(inventory, "price")
print("Products sorted by price:")
for p in sorted_by_price:
    print(p)


# ---------- SORT BY PRODUCT ID (for searching) ----------
sorted_by_id = merge_sort(inventory, "product_id")

# ---------- SEARCH PRODUCT ----------
index = binary_search(sorted_by_id, 103, "product_id")

if index != -1:
    print("\nProduct found:")
    print(sorted_by_id[index])
else:
    print("\nProduct not found")
```

```
Products sorted by price:
105 | USB Cable | Price:150 | Qty:200
102 | Mouse | Price:300 | Qty:150
101 | Keyboard | Price:700 | Qty:50
103 | Monitor | Price:9000 | Qty:30
104 | Laptop | Price:55000 | Qty:10

Product found:
103 | Monitor | Price:9000 | Qty:30
```

Explaination : The program uses Merge Sort to organize the product inventory by attributes like price or quantity, and then applies Binary Search on the sorted list to quickly find a product by its ID or name. This improves performance when dealing with large numbers of products.

Justification : Since the system handles thousands of records, Merge Sort (O(n log n)) efficiently sorts large datasets, while Binary Search (O(log n)) enables very fast product lookup compared to Linear Search (O(n)), meeting the system's performance needs.

Task description #4: Smart Hospital Patient Management System

A hospital maintains records of thousands of patients with details such as patient ID, name, severity level, admission date, and bill amount. Doctors and staff need to:

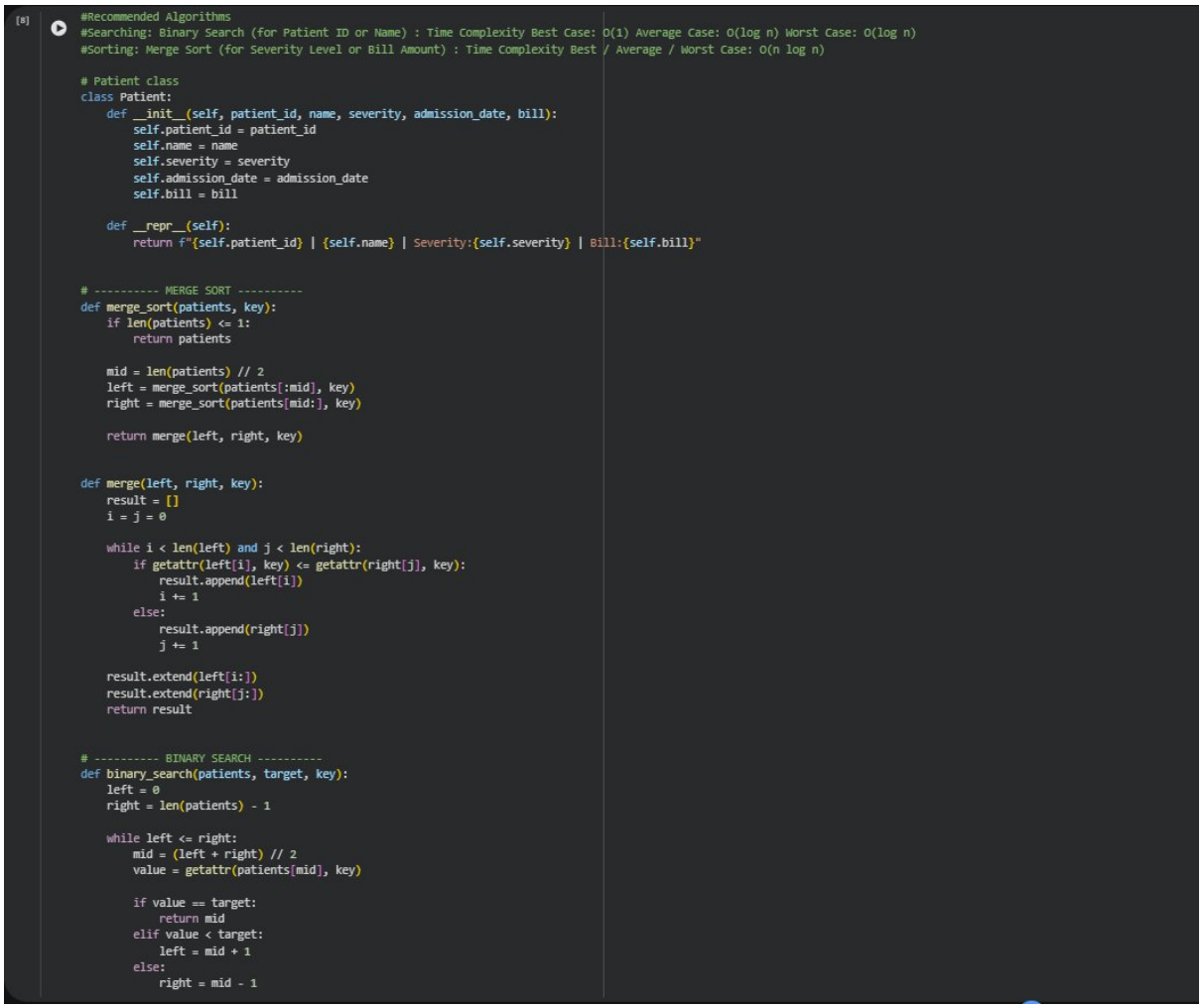
1. Quickly search patient records using patient ID or name.
2. Sort patients based on severity level or bill amount for prioritization and billing.

Student Task

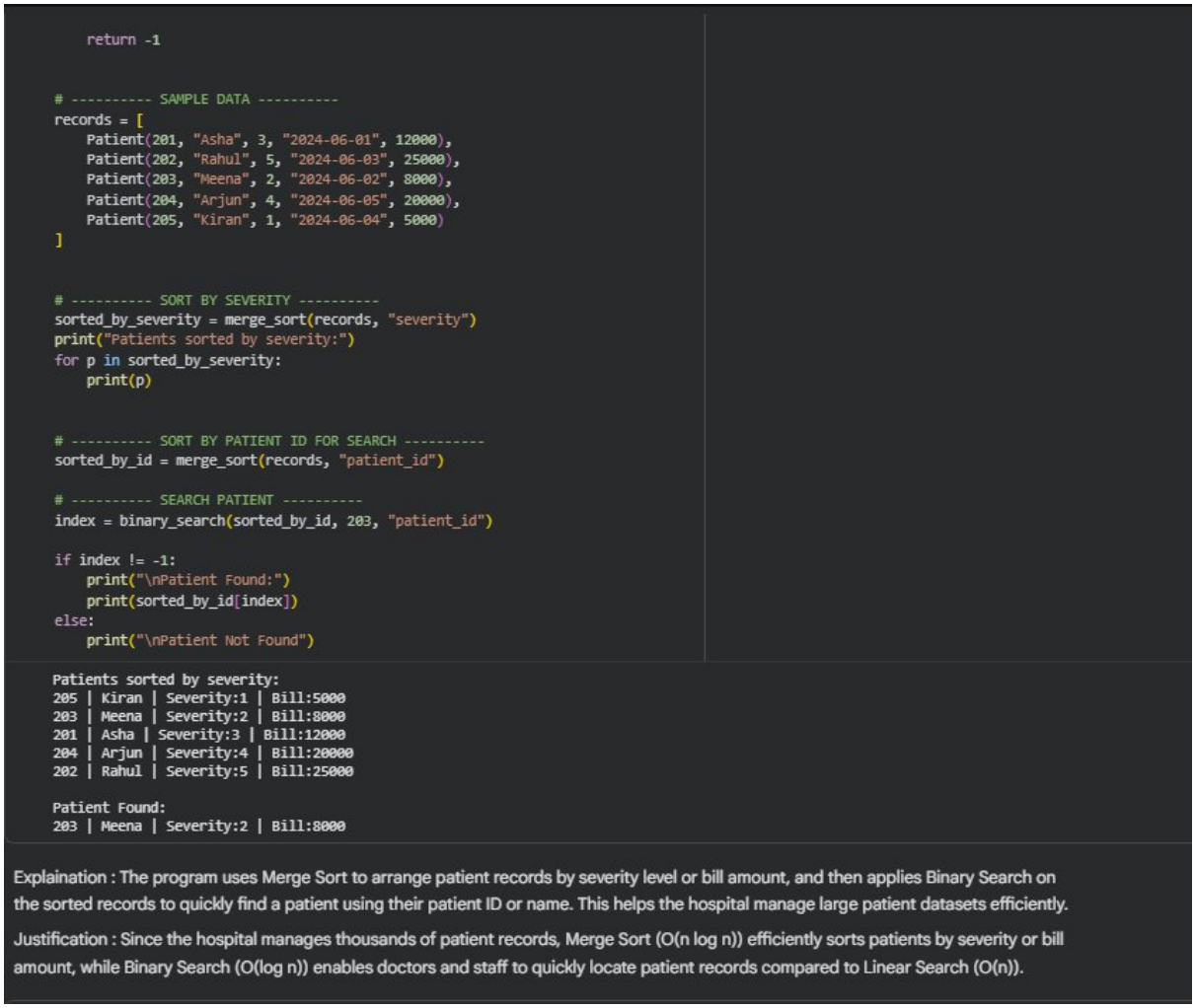
• Use AI to recommend suitable searching and sorting algorithms.

• Justify the selected algorithms in terms of efficiency and suitability.

• Implement the recommended algorithms in Python.



```
[8]
#Recommended Algorithms
#Searching: Binary Search (for Patient ID or Name) : Time Complexity Best Case: O(1) Average Case: O(log n) Worst Case: O(log n)
#Sorting: Merge Sort (for Severity Level or Bill Amount) : Time Complexity Best / Average / Worst Case: O(n log n)

# Patient class
class Patient:
    def __init__(self, patient_id, name, severity, admission_date, bill):
        self.patient_id = patient_id
        self.name = name
        self.severity = severity
        self.admission_date = admission_date
        self.bill = bill

    def __repr__(self):
        return f"{self.patient_id} | {self.name} | Severity:{self.severity} | Bill:{self.bill}"


# ---------- MERGE SORT ----------
def merge_sort(patients, key):
    if len(patients) <= 1:
        return patients

    mid = len(patients) // 2
    left = merge_sort(patients[:mid], key)
    right = merge_sort(patients[mid:], key)

    return merge(left, right, key)


def merge(left, right, key):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if getattr(left[i], key) <= getattr(right[j], key):
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


# ---------- BINARY SEARCH ----------
def binary_search(patients, target, key):
    left = 0
    right = len(patients) - 1

    while left <= right:
        mid = (left + right) // 2
        value = getattr(patients[mid], key)

        if value == target:
            return mid
        elif value < target:
            left = mid + 1
        else:
            right = mid - 1
```



```
    return -1


# ---------- SAMPLE DATA ----------
records = [
    Patient(201, "Asha", 3, "2024-06-01", 12000),
    Patient(202, "Rahul", 5, "2024-06-03", 25000),
    Patient(203, "Meena", 2, "2024-06-02", 8000),
    Patient(204, "Arjun", 4, "2024-06-05", 20000),
    Patient(205, "Kiran", 1, "2024-06-04", 5000)
]


# ---------- SORT BY SEVERITY ----------
sorted_by_severity = merge_sort(records, "severity")
print("Patients sorted by severity:")
for p in sorted_by_severity:
    print(p)


# ---------- SORT BY PATIENT ID FOR SEARCH ----------
sorted_by_id = merge_sort(records, "patient_id")

# ---------- SEARCH PATIENT ----------
index = binary_search(sorted_by_id, 203, "patient_id")

if index != -1:
    print("\nPatient Found:")
    print(sorted_by_id[index])
else:
    print("\nPatient Not Found")
```

```
Patients sorted by severity:
205 | Kiran | Severity:1 | Bill:5000
203 | Meena | Severity:2 | Bill:8000
201 | Asha | Severity:3 | Bill:12000
204 | Arjun | Severity:4 | Bill:20000
202 | Rahul | Severity:5 | Bill:25000

Patient Found:
203 | Meena | Severity:2 | Bill:8000
```

Explaination : The program uses Merge Sort to arrange patient records by severity level or bill amount, and then applies Binary Search on the sorted records to quickly find a patient using their patient ID or name. This helps the hospital manage large patient datasets efficiently.

Justification : Since the hospital manages thousands of patient records, Merge Sort (O(n log n)) efficiently sorts patients by severity or bill amount, while Binary Search (O(log n)) enables doctors and staff to quickly locate patient records compared to Linear Search (O(n)).

Task Description #5: University Examination Result Processing System

A university processes examination results for thousands of students containing roll number, name, subject, and marks. The system must:

1. Search student results using roll number.

2. Sort students based on marks to generate rank lists.

Student Task

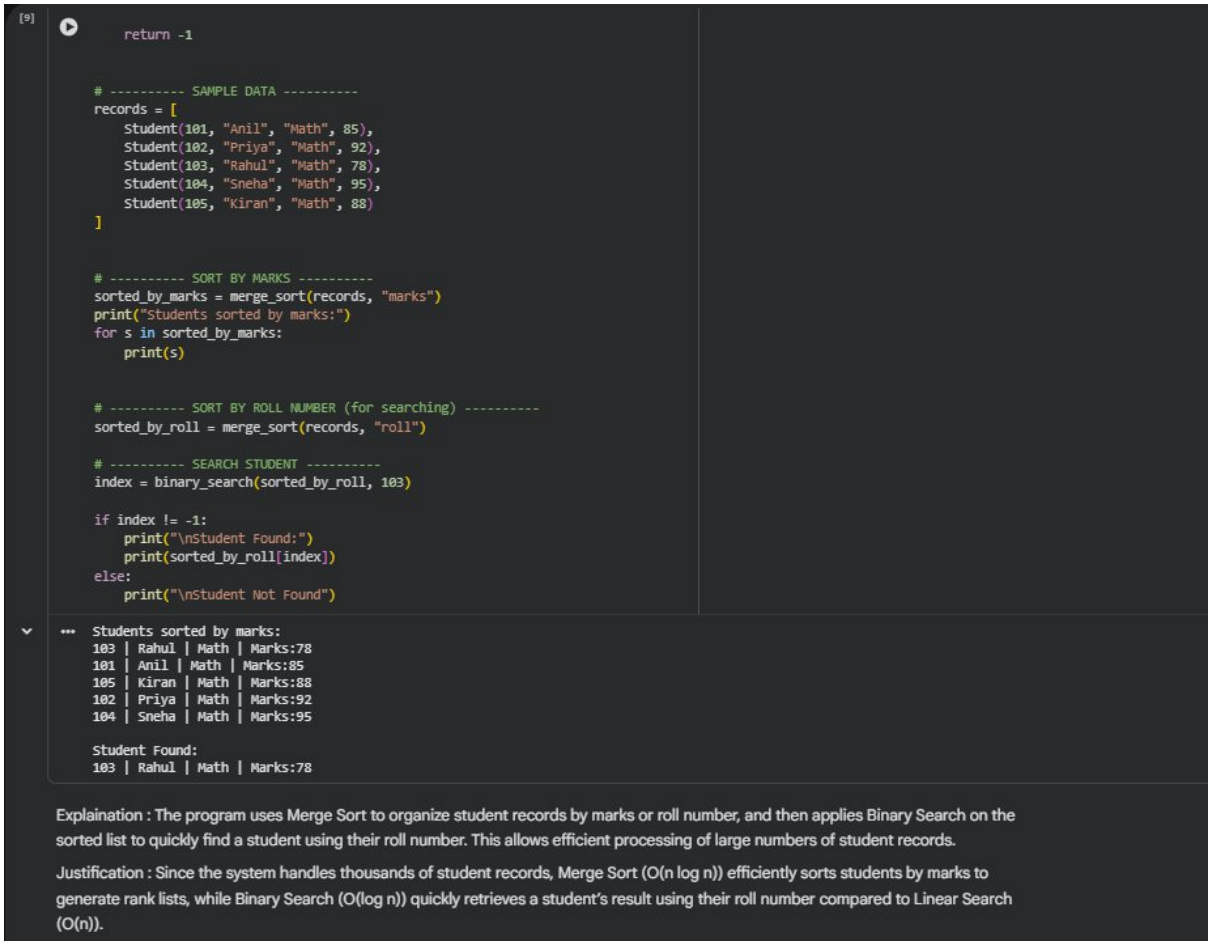
• Identify efficient searching and sorting algorithms using AI assistance.

• Justify the choice of algorithms.

• Implement the algorithms in Python.

[9]

```python
# Searching Algorithm: Binary Search (Used to find a student using their roll number.) : Time Complexity
#Best Case: O(1), Average Case: O(log n), Worst Case: O(log n)
#Sorting Algorithm: Merge Sort (Used to sort students by marks to generate rank lists.) : Time Complexity
#Best Case: O(n log n), Average Case: O(n log n), Worst Case: O(n log n)

# Student class
class Student:
    def __init__(self, roll, name, subject, marks):
        self.roll = roll
        self.name = name
        self.subject = subject
        self.marks = marks

    def __repr__(self):
        return f"{self.roll} | {self.name} | {self.subject} | Marks:{self.marks}"


# ---------- MERGE SORT ----------
def merge_sort(students, key):
    if len(students) <= 1:
        return students

    mid = len(students) // 2
    left = merge_sort(students[:mid], key)
    right = merge_sort(students[mid:], key)

    return merge(left, right, key)


def merge(left, right, key):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if getattr(left[i], key) <= getattr(right[j], key):
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


# ---------- BINARY SEARCH ----------
def binary_search(students, target_roll):
    left = 0
    right = len(students) - 1

    while left <= right:
        mid = (left + right) // 2

        if students[mid].roll == target_roll:
            return mid
        elif students[mid].roll < target_roll:
            left = mid + 1
        else:
            right = mid - 1
```



```
[9]
    return -1


# ---------- SAMPLE DATA ----------
records = [
    Student(101, "Anil", "Math", 85),
    Student(102, "Priya", "Math", 92),
    Student(103, "Rahul", "Math", 78),
    Student(104, "Sneha", "Math", 95),
    Student(105, "Kiran", "Math", 88)
]


# ---------- SORT BY MARKS ----------
sorted_by_marks = merge_sort(records, "marks")
print("Students sorted by marks:")
for s in sorted_by_marks:
    print(s)


# ---------- SORT BY ROLL NUMBER (for searching) ----------
sorted_by_roll = merge_sort(records, "roll")

# ---------- SEARCH STUDENT ----------
index = binary_search(sorted_by_roll, 103)

if index != -1:
    print("\nStudent Found:")
    print(sorted_by_roll[index])
else:
    print("\nStudent Not Found")
```

```
Students sorted by marks:
103 | Rahul | Math | Marks:78
101 | Anil | Math | Marks:85
105 | Kiran | Math | Marks:88
102 | Priya | Math | Marks:92
104 | Sneha | Math | Marks:95

Student Found:
103 | Rahul | Math | Marks:78
```

Explaination : The program uses Merge Sort to organize student records by marks or roll number, and then applies Binary Search on the sorted list to quickly find a student using their roll number. This allows efficient processing of large numbers of student records.

Justification : Since the system handles thousands of student records, Merge Sort (O(n log n)) efficiently sorts students by marks to generate rank lists, while Binary Search (O(log n)) quickly retrieves a student's result using their roll number compared to Linear Search (O(n)).

Task Description #6: Online Food Delivery Platform

An online food delivery application stores thousands of orders with order ID, restaurant name, delivery time, price, and order status. The platform needs to:

1. Quickly find an order using order ID.
2. Sort orders based on delivery time or price.

Student Task

• Use AI to suggest optimized algorithms.

• Justify the algorithm selection.

• Implement searching and sorting modules in Python.

[10] ✓ 0s

```python
#Recommended Algorithms
#Searching: Binary Search (for Order ID)
#Sorting: Merge Sort (for delivery time or price)

# Order class
class Order:
    def __init__(self, order_id, restaurant, delivery_time, price, status):
        self.order_id = order_id
        self.restaurant = restaurant
        self.delivery_time = delivery_time
        self.price = price
        self.status = status

    def __repr__(self):
        return f"{self.order_id} | {self.restaurant} | Time:{self.delivery_time} | Price:{self.price} | {self.status}"


# -------- MERGE SORT --------
def merge_sort(orders, key):
    if len(orders) <= 1:
        return orders

    mid = len(orders) // 2
    left = merge_sort(orders[:mid], key)
    right = merge_sort(orders[mid:], key)

    return merge(left, right, key)


def merge(left, right, key):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if getattr(left[i], key) <= getattr(right[j], key):
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


# -------- BINARY SEARCH --------
def binary_search(orders, target_id):
    left = 0
    right = len(orders) - 1

    while left <= right:
        mid = (left + right) // 2

        if orders[mid].order_id == target_id:
            return mid
        elif orders[mid].order_id < target_id:
            left = mid + 1
        else:
            right = mid - 1
```

```
[10]
✓ 0s
        return -1


    # -------- SAMPLE DATA --------
    orders = [
        Order(201, "Pizza Hut", 30, 450, "Delivered"),
        Order(202, "Dominos", 25, 300, "Preparing"),
        Order(203, "KFC", 40, 550, "Out for delivery"),
        Order(204, "Burger King", 20, 250, "Delivered"),
        Order(205, "Subway", 35, 400, "Preparing")
    ]


    # -------- SORT BY PRICE --------
    sorted_by_price = merge_sort(orders, "price")
    print("Orders sorted by price:")
    for o in sorted_by_price:
        print(o)


    # -------- SORT BY ORDER ID FOR SEARCH --------
    sorted_by_id = merge_sort(orders, "order_id")

    # -------- SEARCH ORDER --------
    index = binary_search(sorted_by_id, 203)

    if index != -1:
        print("\nOrder Found:")
        print(sorted_by_id[index])
    else:
        print("\nOrder Not Found")
```

```
Orders sorted by price:
204 | Burger King | Time:20 | Price:250 | Delivered
202 | Dominos | Time:25 | Price:300 | Preparing
205 | Subway | Time:35 | Price:400 | Preparing
201 | Pizza Hut | Time:30 | Price:450 | Delivered
203 | KFC | Time:40 | Price:550 | Out for delivery

Order Found:
203 | KFC | Time:40 | Price:550 | Out for delivery
```

Explaination : The program uses Merge Sort to arrange orders by delivery time or price, and then applies Binary Search on the sorted list to quickly locate an order using its order ID. This ensures efficient handling of large numbers of orders in the system.

Justification : Since the system handles thousands of orders, Merge Sort (O(n log n)) efficiently sorts orders by delivery time or price, while Binary Search (O(log n)) quickly finds an order using its ID compared to Linear Search (O(n)).